Renewal_calls.csv:
Index(['Call_ID', 'Call_Direction', 'Co_Ref', 'Call_Date',
'Churn_Category', 'Complaint_Category',
'Customer_Reaction_Category',
'Agent_Renewal_Pitch_Category',
'Customer_Renewal_Response_Category',
'Agent_Response_Category',
'Membership_Renewal_Decision', 'Serious_Complaint',
'Other_Complaint', 'Discussion_on_Price_Increase',
'Renewal_Impact_Due_to_Price_Increase',
'Discount_or_Waiver_Requested',
'Call_Reschedule_Request',
'Agent_Flagged_Membership_Status_Alert',
'Agent_Renewal_Initiation', 'Explicit_Competitor_Mention',
'Unnamed: 20', 'Explicit_Switching_Intent',
'Mentioned_Competitors',
'Price_Switching_Mentioned',
'Competitor_Value_Comparison',
'Competitor_Benefits_Mentioned', 'Topic_Introduced_By',
'Percentage_Price_Increase_Mentioned',
'Monetary_Price_Increase_Mentioned',
'Price_Range_Mentioned',
'Customer_Asked_For_Justification', 'Customer_Response',
'Desire_To_Cancel', 'Discount_Offered',
'Justification_Category',
'Reason_For_Renewal_Category',
'Agent_Response_To_Cancel_Category',
'Argument_That_Convinced_Customer_to_Stay_Category',
'Analysed_Call', 'Call_Number', 'Call_Year'],
dtype='str')

email.csv:

Index(['Co_Ref', 'Time_to_Renewal',
'crm_accreditation_completed',
'crm_timely_completion',
'crm_progress_towards_accreditation',
'crm_delays_in_accreditation',
'crm_contractor_suggested_leave',
'crm_contractor_engagement', 'crm_contractor_sentiment',
'crm_contractor_sentiment_score',

'crm_dts_or_ssip_mentioned',
'crm_customer_payment_intention',
'crm_competitors_mentioned',
'crm_membership_level', 'crm_platform_issues_raised',
'crm_agent_chased_contractor', 'crm_agent_chase_count',
'crm_accreditation_issues',
'crm_membership_overdue',
'crm_auto_renewal_status',
'crm_dissatisified_with_renewal_price',
'crm_customer_complained', 'crm_refund_mentioned',
'crm_negative_customer_experience',
'crm_dissatisfaction_with_support',
'crm_financial_hardship_mentioned', 'year'], dtype='str')

cc_calls.csv:

Index(['Contact_ID', 'Call_Date', 'Direction',
'cc_care_package', 'cc_care_package_discussed',
'cc_urgency_getting_on_site', 'cc_external_consultant',
'cc_agent_cross_sell_attempt',
'cc_customer_issues_concerns',
'cc_business_struggles_financial_hardship',
'cc_call_initiated_by', 'cc_questionnaire_completion',
'cc_chasing_response',
'cc_issues_within_questionnaire', 'cc_login_issues',
'cc_platform_issues', 'cc_dissatisfaction_time_to_complete',
'cc_process_complexity_concerns',
'cc_questions_harder_than_expected',
'cc_dissatisfaction_support', 'cc_contractor_sentiment',
'cc_contractor_sentiment_start_score',
'cc_contractor_sentiment_end_score',
'cc_contractor_sentiment_overall_score',
'cc_contractor_sentiment_issues_score',
'cc_pricing_mentioned', 'cc_pricing_sentiment_impact',
'cc_refund_discussed', 'cc_contractor_suggest_leave',
'cc_contractor_complained', 'Co_Ref', 'Analysed_Call',
'Call_Year'], dtype='str')

billings.csv:

Index(['Co_Ref', 'Renewal_Month', 'Connection_Net',
'Connection_Qty', 'Discount_Amount',
'Sustainability_Score', 'Total_Renewal_Score_New',
'Starting_Connection_Net', 'Starting_Connection_Qty',
'Last_Years_Price', 'Last_Years_Date_Paid',
'Auto_Renewal_Score', 'Status_Scores',

'Anchoring_Score', 'Tenure_Scores', 'Proforma_Auto_Renewal', 'Proforma_World_Pay_Token', 'Proforma_Date', 'Current_Anchorings', 'Current_Anchor_List', 'Payment_Timeframe', 'Registration_Date', 'Proforma_Account_Stage', 'Proforma_Audit_Status', 'Current_Auto_Renewal_Flag', 'Current_World_Pay_Token', 'Renewal_Score_At_Release', 'Proforma_Membership_Status', 'Proforma_Approved_Lists', 'Tenure_Years', 'Band', 'Prospect_Renewal_Date', 'Closed_Date', 'Prospect_Status', 'Starting_Net', 'Starting_Vat', 'Starting_Gross', 'Starting_Membership_Net', 'Starting_Package_Net', 'Starting_PQQ_Net', 'Gross', 'Membership_Net', 'Package_Net', 'PQQNet', 'Total_Net_Paid', 'Prospect_Outcome', 'Payment_Method', 'Amount', 'Total_Amount', 'Connection_Group', 'Tenure_Group', '#_of_Connection', 'Last_Renewal', 'Last_Band', 'Last_Total_Net_Paid', 'Last_Connections', 'Anchor_Group', 'Renewal_Year', 'DateTime_Out'], dtype='str')

CSV Datasets: Billings, emails, cc_calls, renewal_calls

Using the 3 datasets build a churn model that identifies if the customer would churn before the subscription due date. Billings data has the general information about the customer. The pre renewal features should be extracted from the renewal calls dataset throught the call_date column (i.e, the call_date should be before the prospect_renewal_date).

The expectation is to follow a proper data science workflow:

Define hypotheses
Perform EDA
Do feature engineering, feature selection
Build and evaluate models
Model explainability
Prepare a PPT and documentation
Final deliverables should include:

Notebooks
Presentation (PPT)
Documentation (Word document)

First start by helping us understand the fields and target variable for the model , and how to train. how to make hypotheses and all.

Great—this is a rich, real-world churn prediction problem. Let's structure it clearly so you can build a solid data science workflow from the ground up.

---

1. 🎯 Define the Business Problem

You want to predict:

👉 **Will a customer churn *before* their subscription renewal date?**

This is a **binary classification problem**:

- **1 = Churn (did NOT renew before due date)**
- **0 = Retained (renewed successfully)**

---

2. 🎯 Define the Target Variable

The target will come primarily from `billings.csv` .

Key fields:

- `Prospect_Renewal_Date`
- `Closed_Date`
- `Prospect_Outcome`
- `Status_Scores` / `Proforma_Membership_Status`

Likely Target Definition:

You need to derive:

```
churn = 1 → customer did NOT renew (or churned before renewal date)
churn = 0 → customer renewed
```

Example logic:

- If:
  - `Prospect_Outcome = Lost` OR
  - No payment before `Prospect_Renewal_Date`
    → **Churn = 1**
- If:

- Payment completed (`Closed_Date` exists and ≤ renewal date)
  → **Churn = 0**

👉 You'll need to validate exact business definitions with stakeholders.

---

3. 📃 Understanding Each Dataset

📊 1. Billings (Core Customer + Target)

This is your **main table (fact table)**.

Key feature groups:

👷 Customer profile

- `Tenure_Years` , `Tenure_Group`
- `Band`
- `Connection_Group`
- `#_of_Connection`

💰 Pricing & payment

- `Last_Years_Price`
- `Total_Net_Paid`
- `Discount_Amount`
- `Payment_Method`

📅 Renewal lifecycle

- `Prospect_Renewal_Date`
- `Closed_Date`
- `Last_Renewal`

🧮 Scores (VERY IMPORTANT)

- `Sustainability_Score`
- `Total_Renewal_Score_New`
- `Auto_Renewal_Score`
- `Anchoring_Score`
- `Tenure_Scores`

👉 These are already engineered features → strong predictors

📞 2. Renewal Calls (Pre-renewal behavioral signals)

This dataset is GOLD for churn signals.

Key idea:

👉 Only use calls **before renewal date**

Filter:

Python Run

Key features:

😡 Customer dissatisfaction

- `Churn_Category`
- `Complaint_Category`
- `Serious_Complaint`
- `Other_Complaint`

💸 Price sensitivity

- `Discussion_on_Price_Increase`
- `Discount_or_Waiver_Requested`
- `Price_Range_Mentioned`
- `Customer_Asked_For_Justification`

🏁 Churn intent signals

- `Explicit_Switching_Intent`
- `Desire_To_Cancel`
- `Explicit_Competitor_Mention`

🧠 Agent interaction

- `Agent_Renewal_Pitch_Category`
- `Agent_Response_Category`
- `Discount_Offered`

👉 These must be **aggregated per customer (Co_Ref)**

📧 3. Emails (Customer sentiment + engagement)

Key signals:

😊 Sentiment

- `crm_contractor_sentiment`
- `crm_contractor_sentiment_score`

😤 Complaints

- `crm_customer_complained`
- `crm_negative_customer_experience`
- `crm_dissatisfaction_with_support`

💸 Price concerns

- `crm_dissatisified_with_renewal_price`

⚠️ Risk signals

- `crm_financial_hardship_mentioned`
- `crm_refund_mentioned`

🏃 Churn hints

- `crm_competitors_mentioned`
- `crm_customer_payment_intention`

☎️ 4. CC Calls (Support interactions)

Key signals:

😡 Friction

- `cc_process_complexity_concerns`
- `cc_login_issues`
- `cc_platform_issues`

😤 Dissatisfaction

- `cc_dissatisfaction_support`
- `cc_dissatisfaction_time_to_complete`

💸 Pricing

- `cc_pricing_mentioned`
- `cc_pricing_sentiment_impact`

📉 Sentiment change

- `cc_contractor_sentiment_start_score`
- `cc_contractor_sentiment_end_score`

---

4. 🔗 Data Joining Strategy

All datasets connect via:

👉 `Co_Ref`

Final dataset:

Start with:

```
billings (1 row per customer)
```

Join aggregated:

- renewal_calls → grouped by Co_Ref
- emails → grouped by Co_Ref
- cc_calls → grouped by Co_Ref

---

5. 🧠 Hypothesis Building (VERY IMPORTANT)

Before modeling, define hypotheses like a business analyst.

🔴 Hypothesis 1: Price sensitivity drives churn

- Customers mentioning price increases are more likely to churn

Features:

- `Discussion_on_Price_Increase`
- `crm_dissatisified_with_renewal_price`
- `cc_pricing_mentioned`

🔴 Hypothesis 2: Negative sentiment → churn

- Lower sentiment scores increase churn probability

Features:

- `crm_contractor_sentiment_score`
- `cc_contractor_sentiment_overall_score`

🔴 Hypothesis 3: Complaints increase churn

- Customers with more complaints churn more

Features:

- `Serious_Complaint`
- `crm_customer_complained`
- `cc_contractor_complained`

🔴 Hypothesis 4: Competitor mentions = high churn risk

- Customers mentioning competitors are likely to leave

Features:

- `Explicit_Competitor_Mention`
- `crm_competitors_mentioned`

🔴 Hypothesis 5: Engagement reduces churn

- More interactions = higher retention

Features:

- Number of calls
- Number of emails
- Agent follow-ups

🔴 Hypothesis 6: Discounts prevent churn

- Customers offered discounts churn less

Features:

- `Discount_Offered`
- `Discount_Amount`

---

6. 🛠️ Feature Engineering Strategy

This is where most of the value comes from.

📞 From Renewal Calls (aggregate per Co_Ref)

Examples:

```python
total_calls
num_complaints
num_price_mentions
num_cancel_requests
```

Python Run

📧 From Emails

```
avg_sentiment_score
num_complaints
```

Python Run

☎️ From CC Calls

```
num_support_calls
avg_sentiment_change
```

Python Run

⏳ Time-based features

Python Run

🔥 Behavioral intensity

Python Run

---

7. 🧪 Model Training Strategy

Step 1: Train-test split

Use time-aware split if possible:

Python Run

Step 2: Models to try

Start simple → then go advanced:

Baseline:

- Logistic Regression

Tree models:

- Random Forest
- XGBoost / LightGBM ✅ (best for this type)

Step 3: Evaluation Metrics

Since churn is usually imbalanced:

- ✅ ROC-AUC
- ✅ Precision / Recall
- ✅ F1-score
- ✅ Confusion Matrix

👉 Business focus:

- High recall for churn (catch churners)

---

8. 🔍 Model Explainability

Use:

- Feature importance (tree models)
- SHAP values (best)

Questions to answer:

- Why do customers churn?
- What are top drivers?

---

9. 📊 EDA Plan

Before modeling:

Check:

- Missing values
- Class imbalance
- Feature distributions

Analyze:

- Churn vs non-churn

- Correlations
- Feature importance (early)

---

10. 📦 Final Deliverables Structure

📓 Notebooks

1. **01_EDA.ipynb**
2. **02_Feature_Engineering.ipynb**
3. **03_Modeling.ipynb**
4. **04_Explainability.ipynb**

📊 PPT Structure

1. Business problem
2. Data overview
3. Hypotheses
4. EDA insights
5. Model results
6. Key drivers of churn
7. Recommendations

📄 Documentation (Word)

Include:

- Data dictionary
- Methodology
- Assumptions
- Model details
- Limitations

🚀 Next Step

If you want, I can help you with:

✅ Writing the **target variable code**
✅ Building **feature engineering pipelines (with Python)**
✅ Creating a **starter notebook template**
✅ Designing your **EDA step-by-step with visuals**

Just tell me 👍

↳ ### Key fields: * `Prospect_Renewal_Date` * `Closed_Date` * `Prospect_Outcome` * `Status_Scores` / `Proforma_Membership_Status`

explain about the key fields about target variable

Great—these fields are *critical* because they define your **ground truth (churn vs retain)**. Let's break them down clearly and practically so you know exactly how to use them.

🎯 1. `Prospect_Renewal_Date`

👉 What it is:

The **due date** by which the customer is expected to renew their subscription.

👉 Why it matters:

This is your **reference point (cutoff date)**.

You will:

- Use it to define **churn timing**
- Filter **pre-renewal behavior** (calls, emails, cc_calls)

👉 Key role in target:

- If the customer does NOT renew **by this date → churn risk**
- All features must be created using data **before this date**

🎯 2. `Closed_Date`

👉 What it is:

The date when the renewal was **actually completed (payment/closure)**.

👉 Why it matters:

It tells you **whether the renewal happened on time or late**.

👉 How to interpret:

Scenario	Meaning
`Closed_Date <= Prospect_Renewal_Date`	✅ Renewed on time
`Closed_Date > Prospect_Renewal_Date`	⚠️ Late renewal (possible churn risk)
`Closed_Date is NULL`	❌ No renewal → churn

👉 Role in target:

This is one of your **strongest signals** for churn.

🎯 3. `Prospect_Outcome`

👉 What it is:

Final business outcome of the renewal process.

Typical values might be:

- `Won` → renewed
- `Lost` → churned
- `Pending` / `In Progress`

👉 Why it matters:

This is often the **cleanest churn label** provided by business teams.

👉 Role in target:

Value	Churn Label
`Lost`	1 (Churn)
`Won`	0 (Retained)

👉 Always validate unique values before using.

🎯 4. `Proforma_Membership_Status`

👉 What it is:

Status of the membership during renewal stage.

Examples:

- Active
- Expired
- Cancelled
- Pending

👉 Why it matters:

Gives **state of customer at renewal time**

👉 Role in target:

Helpful for:

- Cross-checking churn
- Handling ambiguous cases

Example:

- `Cancelled` → churn
- `Expired` with no renewal → churn

🎯 5. `Status_Scores`

👉 What it is:

A derived/internal score indicating customer status or likelihood of renewal.

👉 Why it matters:

- Could be a **precomputed churn likelihood**
- Useful as a **feature**, NOT target

⚠️ Important:

👉 Do **NOT** use this to define churn if it already encodes outcome
→ otherwise **data leakage**

🧠 Putting It All Together (Target Logic)

Here's a robust way to define churn:

✅ Recommended Target Definition

```python
churn = 1 if:
    Prospect_Outcome == 'Lost'
    OR Closed_Date is null
    OR Closed_Date > Prospect_Renewal_Date (optional based on business)

churn = 0 if:
```

Python Run

⚠️ Important Considerations

1. Missing Closed_Date

- Usually means:
  → customer never renewed → churn

---

2. Late Renewals

You must decide:

- Treat as churn? ✅ (common in subscription businesses)
- Or treat as retained?

👉 Ask business stakeholders.

---

3. Conflicting Signals

Example:

- `Prospect_Outcome = Won`
- but `Closed_Date` missing

👉 You must:

- Either prioritize `Prospect_Outcome`
- Or clean inconsistencies

🔍 Data Validation Checklist

Before finalizing target:

✔️ Check unique values:

Python Run

✔ Check missing:

Python Run

✔ Compare consistency:

Python Run

🚨 Common Mistakes to Avoid

❌ Using post-renewal data as features
❌ Using `Closed_Date` directly as input feature
❌ Using `Status_Scores` if it leaks outcome
❌ Not aligning data before renewal date

💡 Final Intuition

Think of it like this:

- `Prospect_Renewal_Date` → **deadline**
- `Closed_Date` → **actual action**
- `Prospect_Outcome` → **final label**
- `Proforma_Membership_Status` → **context/state**

🚀 Next Step

If you want, I can:

✅ Write exact **Python code to create the churn column**
✅ Help you **resolve edge cases in your data**
✅ Show how to **validate target correctness visually**

Just tell me 👍

prospect_outcome : won ,opened , churned
proforma_memebership_status: accredited, in progress, blanks ,member only , non memeber
status_scores : 0 ,7 ,8 ,9

this is what the values are for the these fields
now help me understand better

Great—this clarifies a lot. Now we can **precisely define the target variable** and understand how each field contributes without ambiguity.

Let's interpret each field in a **business + modeling context**, then combine them into a clean churn definition.

🎯 1. `Prospect_Outcome`

Values:

- `won`
- `opened`
- `churned`

🧠 Interpretation:

Value	Meaning	For Model
`won`	Customer renewed successfully	✅ Retained (0)
`churned`	Customer did NOT renew	❌ Churn (1)
`opened`	Renewal process started but not completed	⚠️ Ambiguous

🚨 Important Insight:

👉 `Prospect_Outcome` is your **strongest and cleanest churn signal**

⚠️ What to do with `opened`?

This is tricky. It means:

- Customer is in pipeline

- Outcome not finalized

Options:

Option A (Recommended for modeling):
👉 **Exclude** `opened` **rows**

- Keeps dataset clean
- Avoids label noise

Option B:
👉 Treat as churn if:

- No `Closed_Date`
- Past renewal date

🎯 2. `Proforma_Membership_Status`

Values:

- `accredited`
- `in progress`
- `member only`
- `non member`
- `blank`

🧠 Interpretation:

Value	Meaning	Churn Signal
`accredited`	Fully active & compliant member	✅ Strong retention
`member only`	Partial engagement	⚠️ Medium risk
`in progress`	Not completed onboarding	🔴 High churn risk
`non member`	Not an active member	❌ Likely churn
`blank`	Missing info	⚠️ Needs handling

💡 Key Insight:

👉 This is a **state-of-customer feature**, NOT the target

But it strongly predicts churn:

- `in progress + non member` → high churn probability

🎯 3. `Status_Scores`

Values:

- `0, 7, 8, 9`

🧠 Interpretation (likely business meaning):

Score	Meaning
`0`	Very low engagement / inactive
`7–9`	Increasing engagement / health

💡 Insight:

👉 This is a **numerical proxy for customer health**

- Lower score → higher churn risk
- Higher score → retention likelihood

⚠️ Important:

- Use as a **feature**
- Do NOT use to define churn

🧠 Final Target Variable Definition

Now let's combine everything properly.

✅ Clean Target Logic

Python Run

🚫 Handle opened

Recommended:

Python Run

👉 This avoids:

- Label ambiguity
- Noisy training data

🔍 Cross-Validation with Other Fields

Use these fields to **validate your target**, not define it.

Check consistency:

1. Membership status vs churn

Python Run

Expected:

- `accredited` → mostly retained
- `non member` → mostly churn

---

2. Status score vs churn

Python Run

Expected:

- Score 0 → high churn rate
- Score 9 → low churn rate

🚨 Key Modeling Decisions

❗ Should you use `Proforma_Membership_Status` ?

👉 YES

But:

- Encode properly (One-hot or label encoding)

❗ Should you use `Status_Scores` ?

👉 YES

- Treat as numeric or ordinal

❗ Should you use `Prospect_Outcome` ?

👉 ❌ NO (as feature)

👉 ✅ ONLY as target

💡 Final Intuition

Think of your data like this:

- `Prospect_Outcome` → **Final truth (what happened)**
- `Proforma_Membership_Status` → **Customer maturity**
- `Status_Scores` → **Customer health score**

🧪 Example Final Dataset

After cleaning:

Co_Ref	Status_Scores	Membership_Status	Calls_Features	Email_Features	churn
A123	9	accredited	...	...	0
B456	0	non member	...	...	1

🚀 Next Step

Now that target is clear, the next critical step is:

👉 **Filtering only pre-renewal data (VERY IMPORTANT)**

I can help you with:

✅ Exact Python code to:

- Filter calls before renewal date
- Aggregate features
- Merge datasets

✅ OR help you start your **EDA notebook**

Just tell me 👍

So the target variable is only prospect_outcome right? And other three are for predicting it only? So now help me with what features are used to predict , and help with hypotheses definition

Yes—you're thinking about it correctly now.

👉 **Target variable =** `Prospect_Outcome` **(converted to churn: 1/0)**
👉 The other fields (`Proforma_Membership_Status` , `Status_Scores`) are **predictor features**, not targets.

🎯 1. Final Target Setup (Confirmed)

You will create:

Python Run

👉 Drop:

Python Run

🎯 2. What Features Should Be Used?

You have **4 datasets**, and the idea is:

👉 Use **ONLY information available BEFORE renewal date**

🧱 A. Core Features (from Billings)

These describe **who the customer is**

📊 Customer profile

- `Tenure_Years`
- `Tenure_Group`
- `Band`
- `Connection_Group`
- `#_of_Connection`

👉 Why:

- Long tenure → less churn
- Low connections → higher churn

💰 Financial behavior

- `Last_Years_Price`
- `Total_Net_Paid`
- `Discount_Amount`
- `Payment_Method`

👉 Why:

- Price-sensitive customers churn more
- Discount dependency matters

🧮 Scores (VERY POWERFUL)

- `Sustainability_Score`
- `Total_Renewal_Score_New`
- `Auto_Renewal_Score`
- `Anchoring_Score`
- `Tenure_Scores`
- `Status_Scores` ✅

👉 Why:

- These are already predictive signals

📃 Membership state

- `Proforma_Membership_Status` ✅

👉 Why:

- Strong indicator of maturity and engagement

📞 B. Renewal Calls Features (Pre-renewal ONLY)

👉 Aggregate per `Co_Ref`

🔴 Churn intent signals (MOST IMPORTANT)

- `Explicit_Switching_Intent`
- `Desire_To_Cancel`
- `Customer_Renewal_Response_Category`

👉 Direct signals of churn

💸 Price sensitivity

- `Discussion_on_Price_Increase`
- `Discount_or_Waiver_Requested`
- `Price_Range_Mentioned`

😡 Complaints

- `Serious_Complaint`
- `Complaint_Category`

🧠 Agent influence

- `Discount_Offered`
- `Agent_Response_Category`
- `Argument_That_Convinced_Customer_to_Stay_Category`

📊 Aggregations to create:

```
num_calls
num_complaints
num_price_mentions
```

Python Run

📧 C. Email Features

👉 Aggregate per `Co_Ref`

😊 Sentiment

- `crm_contractor_sentiment_score`

😡 Complaints

- `crm_customer_complained`
- `crm_negative_customer_experience`

💸 Price dissatisfaction

- `crm_dissatisified_with_renewal_price`

⚠️ Risk signals

- `crm_financial_hardship_mentioned`
- `crm_refund_mentioned`

🏁 Churn signals

- `crm_competitors_mentioned`
- `crm_customer_payment_intention`

☎️ D. CC Calls Features

😤 Friction

- `cc_process_complexity_concerns`

- `cc_login_issues`
- `cc_platform_issues`

😡 Dissatisfaction

- `cc_dissatisfaction_support`

💸 Pricing

- `cc_pricing_mentioned`

📉 Sentiment change

- `cc_contractor_sentiment_overall_score`

🧠 3. Feature Categories (Important for thinking)

---

1. Customer Profile

Who they are
→ tenure, band, connections

---

2. Financial Behavior

How they pay
→ price, discounts

---

3. Engagement

How much they interact
→ calls, emails, support

---

4. Sentiment

How they feel
→ sentiment scores

5. Risk Signals

What they say/do
→ complaints, cancellations, competitors

🔬 4. Hypothesis Definition (VERY IMPORTANT)

Now let's define **clear, testable hypotheses**.

🔴 Hypothesis 1: Price sensitivity drives churn

👉 Customers discussing price increases are more likely to churn

Features:

- `Discussion_on_Price_Increase`
- `crm_dissatisified_with_renewal_price`
- `cc_pricing_mentioned`

🔴 Hypothesis 2: Negative sentiment increases churn

👉 Lower sentiment → higher churn

Features:

- `crm_contractor_sentiment_score`
- `cc_contractor_sentiment_overall_score`

🔴 Hypothesis 3: Complaints lead to churn

👉 More complaints = higher churn probability

Features:

- `Serious_Complaint`
- `crm_customer_complained`
- `cc_contractor_complained`

🔴 Hypothesis 4: Explicit churn intent predicts churn

👉 If customer mentions cancel/switch → very high churn

Features:

- `Desire_To_Cancel`
- `Explicit_Switching_Intent`

🔴 Hypothesis 5: Competitor mentions increase churn

👉 Customers comparing competitors are likely to leave

Features:

- `Explicit_Competitor_Mention`
- `crm_competitors_mentioned`

🔴 Hypothesis 6: Low engagement leads to churn

👉 Less interaction → higher churn

Features:

- Number of calls
- Number of emails
- `crm_agent_chase_count`

🔴 Hypothesis 7: Discounts reduce churn

👉 Customers offered discounts are more likely to stay

Features:

- `Discount_Offered`
- `Discount_Amount`

🔴 Hypothesis 8: Customer maturity reduces churn

👉 Mature customers churn less

Features:

- `Tenure_Years`
- `Proforma_Membership_Status`

- `Status_Scores`

🔴 Hypothesis 9: Process friction increases churn

👉 Difficult processes push customers away

Features:

- `cc_process_complexity_concerns`
- `cc_login_issues`

🧪 5. How You Will Test These Hypotheses (EDA)

Examples:

📊 Price vs churn

Python Run

📊 Sentiment vs churn

Python Run

📊 Complaints vs churn

Python Run

🚀 Final Summary

✅ Target:

- `Prospect_Outcome` → churn label

✅ Features:

From:

- Billings (profile + financial + scores)
- Renewal calls (behavior)
- Emails (sentiment)
- CC calls (support experience)

✅ Hypotheses:

- Price, sentiment, complaints, engagement, competitors, discounts

---

🔥 Next Step

Now the most important step is:

👉 **Feature engineering (aggregation + filtering before renewal date)**

I can help you with:

✅ Exact Python code to build final modeling dataset
✅ Aggregation logic for all 3 datasets
✅ Full starter notebook

Just tell me 👍

is it possible to join as one table on the important fields

Yes—it's not only possible, it's exactly what you **should do** 👍
But the key is: **you don't directly join raw tables—you join *aggregated* tables**.

Let me explain this clearly.

🎯 1. Final Goal

You want **ONE row per customer** (`Co_Ref`) like this:

Co_Ref	Features from Billings	Calls Features	Email Features	CC Features	churn

👉 This becomes your **modeling dataset**

⚠️ 2. Why You CANNOT Directly Join Raw Tables

Because:

- `billings` → 1 row per customer ✅
- `renewal_calls` → many rows per customer ❌
- `emails` → many rows per customer ❌
- `cc_calls` → many rows per customer ❌

👉 If you join directly:

- You'll create **duplicate rows**
- Your model will be wrong

✅ 3. Correct Approach (VERY IMPORTANT)

Step 1: Start with Billings

This is your **base table**

Python Run

Step 2: Filter Pre-Renewal Data

For all other datasets:

👉 Only keep records **before renewal date**

Example (renewal calls):

```python
renewal_calls = renewal_calls.merge(
    billings[['Co_Ref', 'Prospect_Renewal_Date']],
    on='Co_Ref',
    how='left'
)
```

```
renewal_calls = renewal_calls[
```

Python Run

👉 Do this for:

- renewal_calls
- cc_calls
- emails (if time column exists)

Step 3: Aggregate Each Dataset

📞 Renewal Calls → Aggregate

```
calls_agg = renewal_calls.groupby('Co_Ref').agg({
    'Call_ID': 'count',
    'Serious_Complaint': 'sum',
    'Discussion_on_Price_Increase': 'sum',
    'Explicit_Switching_Intent': 'sum',
    'Desire_To_Cancel': 'sum',
```

Python Run

Rename columns:

```
calls_agg.columns = [
    'Co_Ref',
    'num_calls',
    'num_serious_complaints',
    'num_price_mentions',
    'num_switch_intent',
    'num_cancel_requests',
```

Python Run

📧 Emails → Aggregate

```
email_agg = emails.groupby('Co_Ref').agg({
    'crm_contractor_sentiment_score': 'mean',
    'crm_customer_complained': 'sum',
    'crm_dissatisified_with_renewal_price': 'sum',
```

Python Run

☎️ CC Calls → Aggregate

```python
cc_agg = cc_calls.groupby('Co_Ref').agg({
    'Contact_ID': 'count',
    'cc_dissatisfaction_support': 'sum',
    'cc_pricing_mentioned': 'sum',
    'cc_process_complexity_concerns': 'sum',
```

Python Run

🔗 Step 4: Join Everything

Now you can safely merge:

```python
df_final = df_base.merge(calls_agg, on='Co_Ref', how='left')
```

Python Run

🎯 Step 5: Add Target Variable

```python
df_final = df_final[df_final['Prospect_Outcome'].isin(['won', 'churned'])]

df_final['churn'] = df_final['Prospect_Outcome'].map({
    'won': 0,
```

Python Run

🧠 4. Important Design Decisions

✅ Use LEFT JOIN

Python Run

👉 Why:

- Keep all customers
- Missing interactions = valuable signal

✅ Handle Missing Values

After merge:

Python Run

👉 Meaning:

- No calls → 0
- No complaints → 0

⚠️ Watch for Data Leakage

Do NOT use:

- `Closed_Date`
- Any post-renewal data

📊 5. Final Dataset Structure

You'll get something like:

Co_Ref	Tenure	Status_Scores	num_calls	complaints	sentiment	churn

---

💡 Key Insight

👉 The model learns from:

- **Who the customer is** (billings)
- **What they said** (calls, emails)
- **How they behaved** (complaints, pricing, sentiment)

🚀 Final Answer

- ✔ Yes, you should combine everything into **one table**
- ✔ But only **after aggregation**
- ✔ Join using `Co_Ref`
- ✔ Filter data **before renewal date**

🔥 Next Step

If you want, I can:

- ✅ Write a **full end-to-end pipeline code**
- ✅ Help you **select best features automatically**
- ✅ Show **EDA visualizations to validate hypotheses**

Just tell me 👍

why before renewal date? Cant they churn after that and before closed date?
correct me if i'm wrong

You're asking exactly the right question—and your intuition is valid. Let's clarify it properly, because this is one of the **most important concepts in churn modeling**.

🎯 Short Answer

👉 Yes, customers *can* churn after the renewal date and before the closed date.
👉 BUT for modeling, we **only use data before the renewal date**.

🧠 Why? (Core Principle)

Because your model should simulate a **real-world prediction scenario**:

> "Based on what we knew *before the renewal deadline*, can we predict if the customer will churn?"

⏳ Timeline View (Very Important)

Let's visualize:

```
--------|------------------|------------------|
   Past     Renewal Date        Closed Date
            (Decision Point)    (Outcome Recorded)
```

🎯 What happens when:

✅ Before `Prospect_Renewal_Date`

- Calls, complaints, emails
- Customer thinking about renewing or not

  👉 This is **valid predictive data**

⚠️ Between Renewal Date and Closed Date

- Follow-ups
- Final negotiations
- Payment processing

👉 This is **too late for prediction**

🚨 The Key Issue: Data Leakage

If you use data **after the renewal date**, you are:

👉 Giving the model **future information**

This leads to:

- Unrealistically high accuracy ❌
- Model that **fails in production** ❌

🔴 Example of Leakage

Customer:

- Renewal Date: Jan 1
- Call on Jan 5: "I already cancelled"
- Closed Date: Jan 10

If you include that Jan 5 call:

👉 The model learns:

> "If customer says they cancelled → churn"

Which is obvious—but useless in real life.

🎯 What You Actually Want

A model that predicts:

> "This customer is *likely to churn* BEFORE they actually churn"

🧠 Business Perspective

Think like a business:

❌ Wrong Approach

Use all data until `Closed_Date`

👉 Prediction happens too late
👉 No time to intervene

✅ Correct Approach

Use only data before `Prospect_Renewal_Date`

👉 Predict churn early
👉 Take action:

- Offer discount
- Call customer
- Resolve issues

🤔 Your Question: Is there churn after renewal date?

Yes—but that's already **post-decision behavior**.

Key distinction:

Stage	Meaning
Before renewal date	Customer is deciding

Stage	Meaning
After renewal date	Outcome is unfolding

🎯 Final Rule

👉 **Features = Before Renewal Date**
👉 **Target = After Renewal Date (final outcome)**

🧪 When *would* you use post-renewal data?

Only if your goal changes to:

- Predict **late payment**
- Predict **drop-off after renewal**
- Predict **customer lifetime churn**

👉 But NOT for this use case

💡 Intuition (Simple)

Think of it like an exam:

- Features = what student studied before exam
- Target = result after exam

You wouldn't include:

> "student saw answer sheet after exam"

🚀 Final Answer

✔️ You are correct that events happen after renewal
❌ But we intentionally ignore them for modeling

👉 To avoid leakage and build a **real-world usable model**

🔥 Next Step

If you want, I can:

✅ Show a **real example of leakage vs correct model**
✅ Help you implement **date filtering properly in code**
✅ Visualize timeline for your dataset

Just tell me 👍

ok tell me the preprocessing and feature engineering

Great—now you're at the **most important stage of the project**. I'll lay this out like a real production-grade churn pipeline so you can directly implement it.

🧠 OVERALL PIPELINE

You will build:

```
Raw Data
   ↓
Data Cleaning
   ↓
Target Creation
   ↓
Preprocessing (types, missing, encoding)
   ↓
Feature Engineering (aggregation + behavioral features)
   ↓
Final Modeling Table (1 row per Co_Ref)
```

🧹 1. DATA PREPROCESSING

📌 1.1 Standardize Column Types

Convert dates:

```python
billings['Prospect_Renewal_Date'] = pd.to_datetime(billings['Prospect_Renewal_Dat
```

Python Run

📌 1.2 Clean target

```python
df = billings[billings['Prospect_Outcome'].isin(['won', 'churned'])]

df['churn'] = df['Prospect_Outcome'].map({
    'won': 0,
```

Python Run

📌 1.3 Handle missing values

Categorical:

Python Run

Numeric:

Python Run

📌 1.4 Remove leakage columns ❌

DO NOT USE:

- `Closed_Date`
- any post-renewal outcome fields
- final status fields that encode churn directly

⏳ 2. TIME FILTERING (VERY IMPORTANT)

For ALL interaction datasets:

Rule:

```
Keep only records where Call_Date < Prospect_Renewal_Date
```

Example:

```python
renewal_calls = renewal_calls.merge(
    billings[['Co_Ref', 'Prospect_Renewal_Date']],
    on='Co_Ref',
    how='left'
)

renewal_calls = renewal_calls[
```

Python Run

Do same for:

- cc_calls
- emails (if date exists)

🔧 3. FEATURE ENGINEERING

We will build features in 4 categories:

👱 A. CUSTOMER PROFILE FEATURES (from Billings)

No aggregation needed.

Keep as-is:

- Tenure_Years
- Band
- Connection_Group
- #_of_Connection
- Status_Scores
- Proforma_Membership_Status

Encoding:

```python
One-hot encode:
- Band
```

Python Run

💰 B. FINANCIAL FEATURES

Create new features:

Python Run

Key idea:

👉 Normalize pricing → makes model more stable

📞 C. RENEWAL CALLS (AGGREGATION FEATURES)

Group by `Co_Ref`

🔥 Count-based features

```
num_calls
num_complaints
num_price_discussions
num_cancel_requests
```

Python Run

🔥 Binary flags (very powerful)

```
ever_complained = max(Serious_Complaint)
```

Python Run

🔥 Intensity features

Python Run

Python Run

```
price_pressure_rate = num_price_discussions / num_calls
```

📧 D. EMAIL FEATURES

Aggregate per Co_Ref

Sentiment

Python Run

Risk signals

```
num_financial_hardship = sum(...)
```

Python Run

Engagement

Python Run

☎️ E. CC CALL FEATURES

Friction signals

```
num_platform_issues
```

Python Run

Sentiment

Python Run

Engagement

Python Run

🔗 4. MERGING ALL FEATURES

Start from billings:

Python Run

Merge:

```
final_df = final_df.merge(calls_agg, on='Co_Ref', how='left')
```

Python Run

🧠 5. FINAL CLEANING

Fill missing values

Python Run

Encode categoricals

```python
pd.get_dummies(final_df, columns=[
    'Band',
    'Proforma_Membership_Status',
```

Python Run

Remove IDs

Python Run

🎯 FINAL FEATURE SET (WHAT MODEL SEES)

Your model will use:

👷 Customer features

- Tenure
- Band
- Membership status
- Connections

💰 Financial features

- Price per connection
- Discount ratio

📞 Call behavior

- complaints
- cancellations
- competitor mentions
- price discussions

📧 Email sentiment

- avg sentiment

- financial stress
- dissatisfaction

☎️ Support friction

- issues raised
- sentiment drop

🧠 FEATURE ENGINEERING HYPER-SUMMARY

You are building:

> "How risky is this customer based on behavior BEFORE renewal?"

🚀 6. BEST PRACTICE ADD-ONS (HIGH IMPACT)

If you want to boost model performance:

🔥 1. Interaction intensity

Python Run

🔥 2. Risk score

```python
risk_score = (
    complaints +
    competitor_mentions +
    negative_sentiment +
```

Python Run

🔥 3. Engagement score

Python Run

🎯 FINAL RESULT

You end with:

```
1 row = 1 customer
columns = structured + behavioral + sentiment + financial
target = churn (0/1)
```

🚀 If you want next step

I can help you with:

✅ Full Python pipeline (ready-to-run notebook)
✅ Feature importance + SHAP plan
✅ Model training (XGBoost tuned)
✅ PPT structure (slide-by-slide)

Just tell me 👍

wait take a step back, i have did till transformation to date type , now help to make hypotheses first and how to combine fields

Good—this is exactly the right moment to slow down and design things properly. If you rush into merging, you'll lose clarity.

Let's do this in the correct order:

1. **Hypotheses first (what we believe drives churn)**
2. **Then mapping fields to each hypothesis**
3. **Then only think about how to combine data**

🧠 STEP 1: WHAT IS A HYPOTHESIS HERE?

A churn hypothesis is simply:

> "A specific customer behavior or attribute increases/decreases the probability of churn."

We will group them into **business logic buckets**.

🔴 HYPOTHESIS SET (CORE FOR YOUR PROJECT)

💰 1. PRICE SENSITIVITY DRIVES CHURN

Hypothesis:

Customers who discuss price increases or request discounts are more likely to churn.

Fields involved:

Renewal Calls:

- `Discussion_on_Price_Increase`
- `Discount_or_Waiver_Requested`
- `Price_Range_Mentioned`

CC Calls:

- `cc_pricing_mentioned`
- `cc_pricing_sentiment_impact`

Emails:

- `crm_dissatisified_with_renewal_price`

Expected pattern:

- Higher values → higher churn probability

😡 2. NEGATIVE SENTIMENT DRIVES CHURN

Hypothesis:

Customers with negative sentiment in calls/emails are more likely to churn.

Fields:

Emails:

- `crm_contractor_sentiment`
- `crm_contractor_sentiment_score`

CC Calls:

- `cc_contractor_sentiment_overall_score`
- `cc_contractor_sentiment_end_score`

Derived idea:

- Sentiment drop = end − start (CC calls)

🚨 3. EXPLICIT CHURN INTENT IS STRONGEST SIGNAL

Hypothesis:

If customer explicitly mentions cancellation or switching, churn probability is very high.

Fields:

Renewal Calls:

- `Desire_To_Cancel`
- `Explicit_Switching_Intent`

CC Calls:

- `cc_contractor_suggest_leave`

Insight:

👉 This is usually the **#1 predictive feature**

🏁 4. COMPETITOR MENTION INDICATES CHURN RISK

Hypothesis:

Customers who mention competitors are actively considering leaving.

Fields:

Renewal Calls:

- `Explicit_Competitor_Mention`
- `Mentioned_Competitors`
- `Competitor_Value_Comparison`

Emails:

- `crm_competitors_mentioned`

Pattern:

- Any mention → high churn risk

😤 5. COMPLAINTS INCREASE CHURN

Hypothesis:

More complaints → higher churn probability.

Fields:

Renewal Calls:

- `Serious_Complaint`
- `Complaint_Category`

CC Calls:

- `cc_contractor_complained`
- `cc_dissatisfaction_support`

Emails:

- `crm_customer_complained`
- `crm_negative_customer_experience`

Derived feature:

- total_complaints = sum across channels

🤦 6. LOW ENGAGEMENT → HIGH CHURN

Hypothesis:

Less interaction with company leads to churn.

Fields:

All datasets:

- number of calls
- number of emails
- cc_calls count

Derived:

- total_touchpoints
- engagement_score

💸 7. DISCOUNTS PREVENT CHURN

Hypothesis:

Customers who receive discounts are more likely to stay.

Fields:

Renewal Calls:

- `Discount_Offered`
- `Discount_or_Waiver_Requested`

Billings:

- `Discount_Amount`

Pattern:

- discount offered → lower churn probability

📃 8. CUSTOMER MATURITY REDUCES CHURN

Hypothesis:

Long-term or high-status customers are less likely to churn.

Fields:

Billings:

- `Tenure_Years`
- `Tenure_Group`
- `Status_Scores`
- `Proforma_Membership_Status`

Pattern:

- higher tenure → lower churn

⚙️ 9. PROCESS FRICTION INCREASES CHURN

Hypothesis:

If customer experiences friction, they churn.

Fields:

CC Calls:

- `cc_login_issues`
- `cc_platform_issues`
- `cc_process_complexity_concerns`

Emails:

- `crm_accreditation_delays`

💳 10. FINANCIAL STRESS DRIVES CHURN

Hypothesis:

Financial difficulty increases churn probability.

Fields:

Emails:

- `crm_financial_hardship_mentioned`

Renewal Calls:

- price increase discussion

🧠 STEP 2: HOW TO COMBINE FIELDS (LOGICAL GROUPING)

Now don't think "join tables" yet—think **feature blocks**.

🧱 FINAL STRUCTURE (IMPORTANT)

You will NOT merge randomly.

You will build:

🟦 BLOCK 1: CUSTOMER BASE (Billings)

ChatGPT Log in Sign up for free

🟩 BLOCK 2: CALL BEHAVIOR (aggregated)

Grouped by Co_Ref:

```
- counts (calls, complaints)
- flags (ever_cancelled)
- intensity (complaint rate)
```

🟧 BLOCK 3: EMAIL BEHAVIOR (aggregated)

```
- sentiment avg
- hardship flags
- complaint counts
```

🟥 BLOCK 4: CC CALL BEHAVIOR (aggregated)

```
- friction issues
- sentiment drop
- support complaints
```

🔗 STEP 3: FINAL COMBINATION RULE

Once each block is ready:

```
START = billings (base table)

LEFT JOIN calls_agg ON Co_Ref
LEFT JOIN email_agg ON Co_Ref
LEFT JOIN cc_agg ON Co_Ref
```

⚠️ IMPORTANT CONCEPT (DO NOT MISS THIS)

👉 You are NOT joining raw data
👉 You are joining **behavioral summaries**

Think:

WRONG ❌	RIGHT ✅
row-level merge	aggregated features
raw calls	counts & flags
raw emails	sentiment summaries

🎯 FINAL INTUITION

Your model will learn:

> "Customers who show price stress + complaints + low engagement + competitor mentions BEFORE renewal are likely to churn."

🚀 WHAT YOU SHOULD DO NEXT

Since you are at preprocessing stage, next step is:

👉 Step 1 (do this now)

- Write down these hypotheses in your notebook (as markdown)

👉 Step 2

- Confirm column availability in each dataset

👉 Step 3

- Start building **aggregation plan per hypothesis**

If you want next help

I can guide you through:

✅ Turning these hypotheses into feature code
✅ Building aggregation templates (copy-paste ready)
✅ Designing your EDA section per hypothesis

Just tell me 👍